Programación Web - II

Laboratorio- Experimento 4 | Manizales, Caldas

INGENIERIA EN INFORMATICA

NICOLAS CORDOBA RAMIREZ

10/09/2026

Experimento 4: Con tu servidor nativo del taller corriendo, prueba estas dos peticiones y anota qué código de estado devuelve cada una:

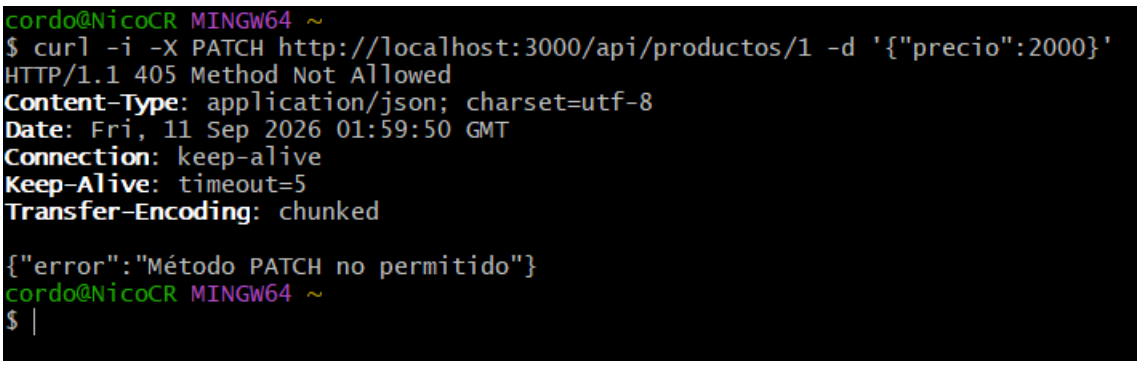
Petición POST: Se ejecutó adaptando la sintaxis de comillas para la terminal de Windows enviando el JSON correspondiente. El servidor retornó **201 Created**

- curl -i -X POST http://localhost:3000/api/productos -H "Content-Type: application/json" -d "{\"nombre\":\"Arepa\",\"precio\":1500,\"stock\":10}"



Petición PATCH: A diferencia de un 404 Not Found (donde la ruta no existe), la respuesta fue un 405 Method Not Allowed. Esto ocurrió porque en servidor.js la ruta sí fue reconocida, pero el servidor nativo detectó que PATCH no esta permitido para el endpoint, el servidor rechaza la solicitud controladamente.

- curl -i -X PATCH http://localhost:3000/api/productos/1 -d '{"precio":2000}'



Cuenta cuántas líneas de tu servidor.js se dedican a *enrutar* y cuántas a la lógica de productos. (Razón de Existir de Express)

- **Líneas para enrutamiento y fontanería: ~22 líneas** (destinadas a extraer la URL, parsear queryParams, evaluar los bloques if (req.method === ...) y responder con estados 404/405).
- **Líneas para la lógica de productos: ~32 líneas** (lectura del stream req.on('data'), parseo JSON, validaciones de negocio y respuesta con la data).

Cerca del 40% del código servidor.js se gasta únicamente en gestionar el enrutamiento básico y el parseo manual de las peticiones.